

RAG vs OKF vs PageIndex, and the other ways to feed your documents to an LLM

Checked against live pages, September 2026. Figures reported by a vendor about its own product are marked (vendor). Figures from a third party are marked (third-party). Anything no research task could confirm says "unverified".

1. Verdict

What the reel got right, and what it left out

The reel is accurate. OKF is real, and it is what the reel says it is. Google Cloud's Data Cloud team announced the Open Knowledge Format on June 12, 2026. It is a directory of Markdown files, one concept per file, each with YAML frontmatter, linked to each other by ordinary Markdown links. Version 0.2 (July 24, 2026) added trust and provenance fields. It is Apache-2.0 and lives at `GoogleCloudPlatform/open-knowledge-format`. Four things the reel does not tell you:

1. **OKF is a file format. It does not retrieve anything.** It has no search and no index beyond an optional `index.md`, and its conformance rules tell consumers to accept broken links. Something else has to find the starting file and decide how many links to follow. In Google's own product, agents do not walk the files at all: they call Knowledge Catalog APIs (`searchEntries`, then `LookupContext`, then `entries.get`).
2. **Google ships no tool that turns a PDF into OKF.** The reference agent's README documents two commands: `enrich --source bq`, which builds a bundle from BigQuery, and `visualize`. Two third-party packages, `google-okf` and `okflib`, claim to do the conversion. `google-okf` is not Google software despite its name. Neither has been tested.
3. **It is young.** The spec is at 0.2, the repo has about 625 stars, and there are no tagged releases.
4. **"OKF" is also the Open Knowledge Foundation**, which has nothing to do with Google. Searches for "OKF" will turn up both.

Decisions you can act on

If your situation is...	Do this
One policy, or a set that fits in context (a few hundred pages at most)	Put the whole thing in the prompt and turn on prompt caching. A 50-page policy costs about \$0.008 per question once cached on Claude Sonnet 5. The problem of a rule on page 2 and its exception on page 15 goes away, because both pages are in the prompt.
One or a few long PDFs, too big to put in the prompt comfortably, and you want page citations without running a vector database	PageIndex

If your situation is...	Do this
Many documents, a growing corpus, or high query volume	Upgraded RAG: chunks split along the document's sections, contextual retrieval, hybrid search (keyword + vector), then a reranker. Add 1-hop cross-reference expansion if testing shows answers missing exceptions.
You write and maintain the knowledge yourself and want a readable, version-controlled source of truth	Store it as OKF, and put one of the options above on top to retrieve from it
Questions about themes across many documents ("what do all these policies say about X")	LightRAG (actively developed, updates incrementally). Use GraphRAG only if you accept that it is in maintenance mode and expensive.
Scanned PDFs, or pages heavy with tables and charts	ColPali-family retrieval over page images, or PageIndex Cloud's OCR
Agent memory where facts change over time	Graphiti/Zep or Cogne. These are memory tools, not document retrieval.
You are weighing RAPTOR	Skip it. The repo has had no commits since September 2024.

For your case (a long policy PDF with exceptions spread across pages): start with the whole document in context plus caching, and run the benchmark in section 6 against PageIndex and an upgraded RAG. Build retrieval only if the document's size or the benchmark results force you to.

2. Feature matrix

2a. Identity and status

Approach	Kind	Maintainer	Licence	Latest verified version	Maturity signals
Vector RAG (baseline)	Technique	Many frameworks	Varies	LlamaIndex 0.14.25; LangChain 1.4.2; Haystack 3.2.0	Default industry pattern
Upgraded RAG (hybrid + rerank + contextual)	Technique	Qdrant, Weaviate, Anthropic, Cohere, Voyage, others	Varies	None (a technique)	Documented by several vendors

Approach	Kind	Maintainer	Licence	Latest verified version	Maturity signals
OKF	Spec (file format)	Google Cloud	Apache-2.0	Spec 0.2; no tagged releases	~625 stars; repo history starts Aug 14, 2026
PageIndex	Library + paid cloud	Vectify AI	MIT	pageindex 0.2.19 (Sep 21, 2026)	~35.9k stars; active
Agentic file search	Pattern	No single maintainer; described by Anthropic and LlamaIndex	None	None (a pattern)	Used in production by Claude Code (per public statements; not from a primary source)
Long context + caching	Vendor feature	Anthropic, OpenAI, Google	Commercial	~1M-token windows on flagship models	Documented by all three vendors
Microsoft GraphRAG	Library	Microsoft Research	MIT	3.2.0	~36.1k stars; maintenance mode
LightRAG	Library	HKU Data Intelligence Lab	MIT	lightrag-hku 1.5.7 (Sep 2, 2026)	~39.9k stars; active; EMNLP 2025
RAPTOR	Research code	Paper author	MIT	No releases, no confirmed PyPI package	~1.8k stars; last commit Sep 3, 2024
Late interaction (ColBERT / ColPali)	Models + libraries	Stanford; Illuin Tech	unverified	colpali-engine deprecated; move to Sentence Transformers v6	ColBERTv2 mature
Memory graphs (Graphiti/Zep; Cognee)	Library + paid cloud	Zep; Topoteretes	Apache-2.0	graphiti-core 0.30.2; cognee 1.6.1	~31k stars each; active

2b. How it works

Approach	What is stored	How retrieval happens	Embeddings / vector DB	LLM calls: ingest / query	Multi-hop, relationships
Vector RAG	Chunk vectors	Top-k similarity	Yes / yes	0 / 1	Weak
Upgraded RAG	Chunks with added context, vectors + keyword index	Hybrid search, fused with RRF, then reranked	Yes / yes	1 per chunk (contextual) / 1 + rerank	Better recall, still no explicit links
OKF	Markdown concept files + YAML frontmatter	Not defined; you add search plus link-following	Optional	Your choice / your choice	Explicit links
PageIndex	Table-of-contents tree (titles, page ranges, summaries)	An LLM reads the tree and picks nodes	No / no	Summaries only (Flash: none for structure) / 1 in the documented base version; production API uses MCTS, undocumented	Via the tree; within one document
Agentic file search	Plain files	Agent loops over grep, glob and read	No / no	0 / several turns	Follows whatever it reads
Long context + caching	Nothing; whole corpus in the prompt	None	No / no	0 / 1	Everything is in view
GraphRAG	Entity graph + community summaries	Local, global (map-reduce), DRIFT, basic	Yes / yes	Many (extraction + summaries) / 1 to many	Strong; also answers questions about the whole corpus
LightRAG	Graph + vectors	naive, local, global, hybrid, mix modes	Yes / yes	1 per chunk / 1+	Strong

Approach	What is stored	How retrieval happens	Embeddings / vector DB	LLM calls: ingest / query	Multi-hop, relationships
RAPTOR	Tree of cluster summaries	Collapsed-tree or tree-traversal similarity search	Yes / yes	1 per cluster / 1	Synthesis at several levels of detail
Late interaction	Vectors per token (ColBERT) or per page-image patch (ColPali)	MaxSim scoring	Multi-vector / yes	0 / 1	Weak, same as RAG
Memory graphs	Graph of facts with time stamps (Graphiti); knowledge graph (Cognee)	Hybrid search + graph	Yes / graph DB	Per episode / 1+	Strong; facts carry dates

2c. Operational properties

Approach	Incremental updates	Citations	Source stays readable	Tables / scans	Corpus ceiling	Main failure mode
Vector RAG	Re-embed changed chunks	Chunk-level, if you store it	Originals yes; index no	Chunking splits tables	Millions (latency roughly flat from 10k to 10M docs)	Finds the rule, misses its exception
Upgraded RAG	Re-process changed chunks	Chunk-level	Originals yes	Better if you split by structure	Millions	Same problem, less often
OKF	Edit the file; CI push (Knowledge Catalog)	sources, generated, verified fields (v0.2)	Yes, by design	Markdown tables	unverified	Stale or broken links, which the spec tolerates

Approach	Incremental updates	Citations	Source stays readable	Tables / scans	Corpus ceiling	Main failure mode
PageIndex	unverified	Page and block citations with bounding boxes	Original PDF yes; index is tree JSON	Local mode: text PDFs only; Cloud: OCR	Scoped to one doc_id per query; no stated limit	unverified (only one error documented)
Agentic file search	Edit the file	Knows the file path	Yes	Plain text only; no PDFs	Hundreds to low thousands of files	Wrong vocabulary, too many turns
Long context + caching	Nothing to update	Ask the model to quote	Yes	Depends on text extraction	~1M tokens (about 1,500 to 2,000 pages)	Cost and latency as size grows
GraphRAG	update CLI; how communities are recomputed is not documented	Local: source text; global: generated reports	No (generated)	unverified	unverified	Cost; project no longer developed
LightRAG	Native , uses its cache	Links back to source chunks	No	unverified	unverified	Slower per query (19.8 to 23.6 s median, third-party)
RAPTOR	None; rebuild	Through the tree to the leaf chunks	No	unverified	unverified	Unmaintained; full rebuilds
Late interaction	Re-embed changed pages	Page-level	Originals yes	Page images, no OCR needed	Heavy storage	Storage and compute cost

Approach	Incremental updates	Citations	Source stays readable	Tables / scans	Corpus ceiling	Main failure mode
Memory graphs	By design	Episodes	No	unverified	unverified	Wrong tool for static documents

2d. Cost and fit

Approach	Ingest cost	Per-query cost	Latency	Free vs paid	Best fit
Vector RAG	Embeddings: \$0.02/1M tokens (OpenAI 3-small, Voyage 4-lite)	1 LLM call	unverified	Free open-source stack; managed databases paid	Large, fuzzy corpora
Upgraded RAG	Plus \$1.02/1M document tokens for contextual retrieval (vendor)	Plus reranker (price unverified)	unverified	Same	Default for many documents
OKF	Your authoring time or LLM splitting (unverified)	Search + one read per hop	unverified	Free spec; Knowledge Catalog is GCP	Knowledge you curate and version
PageIndex	Self-host ~\$0.001/page in LLM tokens (vendor); Cloud \$0.01/page	Your LLM call(s); Cloud \$0.001/page/month to keep pages active	Indexing 13 s to 4.5 min for 9 to 1,098 pages (vendor); per query unverified	Free self-host; Cloud: \$10 credit and 1,000 active pages free	Long structured PDFs
Agentic file search	None	Grows with turns and file sizes	Slower than precomputed retrieval (Anthropic)	Model cost only	Small, well-named file sets
Long context + caching	None	50-page policy on Sonnet 5: ~\$0.053 uncached, ~\$0.008 cached	unverified	Model cost only	Corpora that fit and get queried repeatedly

Approach	Ingest cost	Per-query cost	Latency	Free vs paid	Best fit
GraphRAG	~\$1.81 per 100 docs; \$2.25 for 45k words (third-party)	Local \$5.23 per 1k questions; global \$66.22 per 1k (third-party)	Local 6.5 s median (third-party)	Free, self-host only	Themes across a whole corpus
LightRAG	1 LLM call per chunk; \$ unverified	\$38.67 to \$42.29 per 1k questions (third-party)	19.8 to 23.6 s median (third-party)	Free, self-host only	Relationship questions on a changing corpus
RAPTOR	1 LLM call per cluster; \$ unverified	unverified	unverified	Free	Research only
Late interaction	unverified	unverified	unverified	Free models	Scans, charts, tables
Memory graphs	Per episode	Metered in the cloud	unverified	Zep: free tier, Flex \$125/month; Cognee Cloud: free tier, then \$1/1M tokens	Agent memory

The long-context figures assume a 50-page policy is about 25,000 tokens (about 500 tokens per page, an estimate rather than a measurement), with Sonnet 5 at \$2/1M input and \$10/1M output, a 300-token answer, cache writes at 1.25x the input price and cache reads at 0.1x. The first cached call costs about \$0.065, so caching only pays off from the second question inside the cache lifetime (5 minutes, or 1 hour at a 2x write price).

3. The approaches one by one

3.1 Vector RAG (the baseline)

How it works. Split documents into chunks, embed each chunk, and store the vectors. At query time, embed the question, fetch the top-k nearest chunks, and have the LLM answer from them. Chunks can be fixed-size with overlap, split recursively on separators, split where the topic shifts (semantic), or split along the document's own headings and sections (structure-aware).

When it wins. Large corpora, questions phrased in words the documents don't use, and cheap queries. Nearest-neighbour search latency stays roughly flat from 10k to 10M documents (LlamaIndex).

When it doesn't. When a rule and its exception sit on different pages, similarity search can return one without the other. It also fails on questions about the whole corpus ("naive RAG only considers the top-k most similar chunks", Microsoft Research), and it only knows what was in the last indexing run.

Status. LlamaIndex 0.14.25 (Sep 21, 2026), LangChain 1.4.2 (Sep 18, 2026), Haystack 3.2.0 (Sep 24, 2026), all from PyPI. Vector stores: pgvector 0.8.6 (PostgreSQL licence), Qdrant (Apache-2.0, free cloud tier), Weaviate (BSD-3, free cloud tier), Chroma (Apache-2.0), FAISS (MIT, a library rather than a server), Pinecone (managed only, free Starter tier). Open embedding models: BAAI/bge-m3 (MIT, produces dense, sparse and multi-vector output) and Qwen3-Embedding-8B (Apache-2.0). When choosing a model on the MTEB leaderboard, look at the Retrieval column, not the overall average.

Implement it. `pip install llama-index`, then:

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

documents = SimpleDirectoryReader("data").load_data()
index = VectorStoreIndex.from_documents(documents)
query_engine = index.as_query_engine()
response = await query_engine.aquery("your question here")
```

Source: https://developers.llamaindex.ai/python/framework/getting_started/starter_example/

3.2 Upgraded RAG (hybrid search + reranking + contextual retrieval)

How it works. Four additions, each well documented, that any fair comparison has to credit to RAG:

- **Hybrid retrieval.** Run BM25 (keyword) and dense (vector) search side by side and fuse the two ranked lists. Qdrant defaults to Reciprocal Rank Fusion, which its docs call "the safe default without an evaluation set".
- **Reranking.** A cross-encoder re-scores the top-N results against the query. Current options include Cohere Rerank 3.5 and Rerank 4 (Fast and Pro) and Voyage rerank-2.5 and rerank-3.
- **Contextual retrieval.** An LLM writes a short context for each chunk and prepends it before the chunk is embedded and keyword-indexed. Anthropic reports that top-20 retrieval failures fall 35% with contextual embeddings alone, 49% when contextual BM25 is added, and 67% when reranking is added on top (vendor). The one-time cost is \$1.02 per million document tokens with prompt caching (vendor).
- **Query rewriting and metadata filters.** LangChain's MultiQueryRetriever fans one question out into several rewrites. Pinecone-style filters restrict the search by source or date.

When it wins. Many documents with steady query volume. This should be the default whenever the corpus is too big to put in the prompt.

When it doesn't. It raises recall but still creates no explicit link between a rule and its exception. Questions about themes across the whole corpus remain weak.

Status. These are techniques rather than products. Cohere's per-search rerank price and Voyage's rerank price are unverified; Cohere's pricing page only showed dedicated-instance prices.

Implement it. No single install. Follow Anthropic's contextual retrieval post for ingest, Qdrant's hybrid-queries docs for fusion, and the Cohere or Voyage reranker docs. Sources: <https://www.anthropic.com/news/contextual-retrieval> , <https://qdrant.tech/documentation/concepts/hybrid-queries/> , <https://docs.voyageai.com/docs/reranker>

3.3 OKF (Open Knowledge Format)

How it works. A bundle is a folder of .md files. The only required frontmatter field is type. title, description, resource and tags are recommended. Version 0.2 adds optional provenance and lifecycle fields: sources, generated, verified, status (draft, stable or deprecated) and stale_after. Two file names are reserved: index.md lists what is in a folder so an agent can read the bundle progressively, and log.md holds a dated change history. Links take one of two forms: /tables/customers.md (relative to the bundle root) or ./other.md (relative to the current file). The spec does not define retrieval. You choose how to find the starting file and how many hops to follow.

When it wins. When you write and maintain the knowledge yourself, want it in git, want people and agents reading the same files, and have explicit relationships between concepts (rule, eligibility, exceptions). The stale_after and verified fields suit policies that are reissued every year.

When it doesn't. When you are starting from a PDF and have no tool to convert it (you have to build one). When the corpus is large and nothing indexes it. When the links go stale, which the spec tolerates silently.

Status. Spec 0.2, Apache-2.0, repo last commit Aug 21, 2026, no releases. Posts: v0.1 (June 12, 2026), v0.2 (July 24, 2026), Knowledge Catalog integration (Aug 26, 2026). The third-party packages google-okf 0.1.3 (an individual developer, MIT) and okflib 0.5.0 (Gwenlake) have not been tested.

Implement it. From a clone of the repo, per its README:

```
python3.13 -m venv .venv
.venv/bin/pip install --index-url https://pypi.org/simple/ -e .[dev]
.venv/bin/python -m reference_agent enrich \
  --source bq \
  --dataset <project>.<dataset> \
  --web-seed-file <path/to/seeds.txt> \
  --out ./bundles/<name>
.venv/bin/python -m reference_agent visualize --bundle ./bundles/<name>
```

As the README documents it, enrich reads from BigQuery. For your PDF you would write the concept files yourself or have an LLM draft them. A minimal concept file, following the spec (the fields match the example in Google's post):

```
---
type: policy
title: Leave Rules
description: Annual leave entitlement and how it accrues.
status: stable
stale_after: 2026-12-31T00:00:00Z
---
Employees get 20 days per year. Exceptions: see [exceptions](./exceptions.md).
```

Sources: <https://github.com/GoogleCloudPlatform/open-knowledge-format> ,
<https://github.com/GoogleCloudPlatform/open-knowledge-format/blob/main/SPEC.md>

3.4 PageIndex

How it works. PageIndex turns a document into a tree like a table of contents: each node has a `title`, a `node_id`, a page range, an optional summary written by an LLM, and child nodes. By default the structure comes from the page layout and a basic model only writes the summaries. The "PageIndex Flash" mode (Aug 26, 2026) builds the structure with no LLM at all. At query time, the documented base algorithm puts the whole tree in one prompt and asks the LLM for the IDs of the nodes likely to contain the answer. The hosted API "uses a combination of LLM tree search and value function-based Monte Carlo Tree Search", with details not yet published.

When it wins. Long PDFs with real structure, such as reports, filings and policies. You need no vector database, citations point to specific pages and blocks, and setup is minimal.

When it doesn't. Each query is scoped to one `doc_id`, so it has no cross-library search of its own. Local mode only reads text PDFs; scanned documents need the Cloud service. Incremental re-indexing and per-query latency are undocumented. The 98.7% FinanceBench score is self-reported: it was run by the vendor on its own Mafin 2.5 system.

Status. MIT, v0.2.19 (Sep 21, 2026), ~35.9k stars, last push Sep 24, 2026. Cloud pricing: indexing \$0.01 per page (one-time), \$0.001 per page per month to keep a page active, the first 1,000 active pages free each month, \$10 free credit, and no fee for retrieval calls. The JS SDK's npm version is unverified.

Implement it. `pip install -U pageindex`

```
import os
from pageindex import PageIndexClient

os.environ["OPENAI_API_KEY"] = "your-openai-key"

client = PageIndexClient(
    index="gpt-5.6-luna",          # model to build the tree index
    chat="gpt-5.6-sol",          # model to search the tree
)
doc_id = client.submit_document("report.pdf")["doc_id"]

answer = client.chat("What was the 2023 operating margin?", doc_id=doc_id)
print(answer)
```

Source: <https://github.com/VectifyAI/PageIndex> (also <https://docs.pageindex.ai/getting-started>, which has the Cloud variant with `index="cloud"` and `PAGEINDEX_API_KEY`)

3.5 Agentic file search

How it works. The agent gets `grep`, `glob` and `read` tools and several turns, and explores plain files instead of querying an index. The "agent follows the links" part of the reel is this pattern. Anthropic recommends a hybrid: load an entry file up front and let the agent explore from there. It names legal and finance work as a good fit, and warns that exploring at runtime "is slower than retrieving pre-computed data".

When it wins. Hundreds to low thousands of well-named text files. LlamaIndex reports that file search alone beats naive semantic search on small and medium collections (vendor blog).

When it doesn't. Collections of 1k to 1M+ files. Questions phrased in different words from the documents (for example "revenue recognition" versus "ASC 606"). PDFs, which grep cannot read. Cost grows with every turn.

Status and delivery. MCP (spec version 2026-07-28, with Tier 1 SDKs in TypeScript, Python, C#, Go and Rust) is the standard way to expose files or search tools to any agent. llms.txt is a starting-point file for a website. Neither one retrieves or follows links for you.

Implement it. There is no package. Give the agent file tools over the folder, add an index file that is loaded up front, and use descriptive filenames with explicit links. Sources:

<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents> ,

<https://www.llamaindex.ai/blog/is-grep-all-you-need-lexical-vs-semantic-search-for-agents> ,

<https://modelcontextprotocol.io/docs/sdk>

3.6 Long context + prompt caching

How it works. Put the whole corpus in the prompt and cache it. The research name for this is cache-augmented generation ("Don't Do RAG", arXiv:2412.15605; the paper page was not fetched). Current windows: Claude Opus 5.5 and Sonnet 5 have 1M tokens; the GPT-6 family about 1.05M; Gemini 3.1 Pro and Gemini 3 Flash 1M.

When it wins. A bounded corpus that you query repeatedly. There is no retrieval step, so no retrieval errors and nothing to maintain. For a single policy this is the strongest baseline.

When it doesn't. When the corpus gets close to the context window. When queries are too rare for the cache to be reused before it expires. When documents change so often that you keep paying the cache-write premium.

Status. Caching prices:

- Anthropic: 5-minute cache writes at 1.25x the input price, 1-hour writes at 2x, reads at 0.1x (0.05x on Opus 5.5, 0.025x on Fable 5.1).
- OpenAI: caching is automatic and cached input costs 10% of the normal price. The cache lifetime is unverified.
- Gemini: caching happens automatically on 2.5+ models, with a 4,096-token minimum on 3.x. The cache lifetime is unverified.

Implement it. There is no install beyond the vendor SDK. Put the document in a cached prompt block as the prompt-caching docs describe. No research task captured an official code sample, so none is given here.

Source: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>

3.7 Microsoft GraphRAG

How it works. GraphRAG splits text into units, has an LLM extract entities, relationships and claims, clusters the resulting graph with the Leiden algorithm, and writes summaries of each cluster ("community") from the

bottom up. Query modes: local (entities plus the surrounding text), global (map-reduce over all community reports), DRIFT, and basic vector search.

When it wins. "What are the themes across everything" questions, and reasoning about entities that spans many documents.

When it doesn't. Small corpora, simple fact lookups, and data that changes often. Its own README warns that indexing "can be an expensive operation". The project is in maintenance mode: bug fixes only and no new features, and it is "not an officially supported Microsoft offering".

Status. MIT, v3.2.0 on GitHub and PyPI. Third-party costs: ~\$1.81 to index 100 documents (theneuralbase), \$2.25 and ~48 minutes for 45k words on GPT-3.5 Turbo (FalkorDB). AWS benchmark: local search \$5.23 per 1k questions at 6.5 s median; global search \$66.22 per 1k questions. A widely repeated \$33,000 indexing figure is unverified.

Implement it. `pip install graphrag`, put your text files in the input folder the docs describe, then:

```
graphrag init --root ./ragtest
# edit ./ragtest/settings.yaml: LLM and embedding API key
graphrag index --root ./ragtest
graphrag query --root ./ragtest --method global "your question"
```

Source: <https://microsoft.github.io/graphrag/> (CLI reference: <https://microsoft.github.io/graphrag/cli/>)

3.8 LightRAG

How it works. One LLM call per chunk extracts entities and relationships together. It builds a graph and a vector index, and queries at two levels. Query modes: naive, local, global, hybrid and mix (the default). When a document is updated or deleted, it rebuilds only the affected entities from its LLM cache.

When it wins. Questions about how things relate, over a corpus that changes, without GraphRAG's re-indexing cost. In the AWS benchmark it was more accurate than GraphRAG's local search on MuSiQue and 2Wiki (token-F1 of 0.60 to 0.65).

When it doesn't. When query speed and cost matter: \$38.67 to \$42.29 per 1k questions at 19.8 to 23.6 s median in the same benchmark (third-party). Its indexing cost in dollars is unverified.

Status. MIT, `lightrag-hku` 1.5.7 (Sep 2, 2026), ~39.9k stars, last commit Sep 26, 2026.

Implement it. `pip install lightrag-hku` (or `uv tool install "lightrag-hku[api]"` for the server), set `OPENAI_API_KEY`. From the official example (an excerpt: the full script defines `WORKING_DIR` and runs these calls inside an `async main()`):

```
from lightrag import LightRAG, QueryParam
from lightrag.llm.openai import gpt_4o_mini_complete, openai_embed

rag = LightRAG(
    working_dir=WORKING_DIR,
    embedding_func=openai_embed,
    llm_model_func=gpt_4o_mini_complete,
```

```

)
await rag.initialize_storages()
with open("./book.txt", "r", encoding="utf-8") as f:
    await rag.ainsert(f.read())
await rag.aquery(
    "What are the top themes in this story?", param=QueryParam(mode="naive")
)

```

Source: https://github.com/HKUDS/LightRAG/blob/main/examples/lightrag_openai_demo.py

3.9 RAPTOR

How it works. RAPTOR embeds the chunks, groups them into clusters (Gaussian mixture, so one chunk can belong to several), has an LLM summarise each cluster, then repeats the process on the summaries until one tree remains. At query time it either searches all levels of the tree at once ("collapsed tree", which does better in the paper) or walks down it level by level.

When it wins. Synthesising across a long document at several levels of detail. The paper reports a 20-point gain on QuALITY when paired with GPT-4.

When it doesn't. In production: the code is unmaintained, has no releases and cannot update incrementally.

Status. MIT, ~1.8k stars, last commit Sep 3, 2024. No official PyPI package could be confirmed: the README installs from a git clone, and the raptor-rag PyPI page returned an error when checked.

Implement it. `git clone https://github.com/parthsarathi03/raptor.git && cd raptor && pip install -r requirements.txt`

```

import os
os.environ["OPENAI_API_KEY"] = "your-key"
from raptor import RetrievalAugmentation
RA = RetrievalAugmentation()
RA.add_documents(text)
answer = RA.answer_question(question="Your question here")

```

Source: <https://github.com/parthsarathi03/raptor>

3.10 Late interaction: ColBERT and ColPali

How it works. Instead of one vector per chunk, these models store a vector for every token (ColBERT) or for every patch of a page image (ColPali, ColQwen). Documents are scored with MaxSim: each query token is matched to its most similar document vector, and those best scores are summed. ColPali embeds the page image itself, so no OCR or layout parsing is needed.

When it wins. Scanned PDFs, tables, charts, and layouts that break text extraction. The ViDoRe scores cited on the repo range from 80.1 to 90.9 across variants (from the model authors).

When it doesn't. It needs much more storage and compute than one vector per chunk. It does nothing about links between pages.

Status. ColBERTv2 with PLAID indexing is mature (the repo points to RAGatouille as an easier wrapper). colpali-engine is deprecated in favour of Sentence Transformers v6 MultiVectorEncoder. The licences are unverified.

Implement it. The quickstart is unverified. The repo points to Sentence Transformers v6 and to vector stores that support multiple vectors per item (Qdrant, Weaviate, Elasticsearch). Source: <https://github.com/illuin-tech/colpali>

3.11 Memory graphs: Graphiti/Zep and Cognee

How it works. These build a graph from a stream of events or conversation turns ("episodes"). In Graphiti, each fact records both when it was true and when it was ingested, so a newer fact can replace an old one without losing the history. Cognee turns text into entities, relationships and searchable chunks, and can run on local CPU models with no API key.

When it wins. Agent memory that changes: what a user said last week, and which facts have since been replaced.

When it doesn't. For your policy PDF. These are memory tools, not document retrieval.

Status. graphiti-core 0.30.2 (Apache-2.0, needs Neo4j, FalkorDB, Neptune or Kuzu). Zep pricing: free tier 10k credits a month, Flex \$125 a month. cognee 1.6.1 (Apache-2.0). Cognee Cloud: free tier 1M tokens, Standard \$1 per 1M tokens.

Implement it. `pip install graphiti-core`, then run the quickstart scripts at <https://github.com/getzep/graphiti/tree/main/examples/quickstart>. For Cognee: `uv pip install "cognee[gliner]"`, then `await cognee.remember(...)` and `await cognee.recall(...)` as shown at <https://docs.cognee.ai/quickstart>

4. What else to add

Your three (RAG, OKF, PageIndex) cover chunk similarity, explicit links and document structure. These also belong in the comparison:

Add	Why it earns its place	Link
Long context + caching	The baseline to beat. It removes the retrieval problem entirely for any corpus that fits, and costs about a cent per question for a 50-page policy.	https://platform.claude.com/docs/en/build-with-claude/prompt-caching
Contextual retrieval	Cheap to add at ingest, with the largest reported gain for plain RAG (up to -67% retrieval failures combined with reranking; vendor)	https://www.anthropic.com/news/contextual-retrieval
Hybrid search + reranking	Catches exact terms such as clause numbers and defined terms that vector search misses	https://qdrant.tech/documentation/concepts/hybrid-queries/

Add	Why it earns its place	Link
Agentic file search	The mechanism the reel's "agent follows the links" actually depends on	https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
LightRAG	The maintained graph option, with incremental updates	https://github.com/HKUDS/LightRAG
Microsoft GraphRAG	The reference design for questions about a whole corpus. Include it to know what it costs, not to build on it.	https://github.com/microsoft/graphrag
ColPali family	The option for PDFs where text extraction fails	https://github.com/illuin-tech/colpali
RAPTOR	The other tree approach besides PageIndex. Worth knowing, not worth building on.	https://github.com/parthsa/rthi03/raptor
Graphiti/Zep, Cognee	Neighbouring category: agent memory rather than document retrieval	https://github.com/getzep/graphiti
MCP	How to deliver whichever option you choose to any agent	https://modelcontextprotocol.io
llms.txt	A starting-point file for docs on a website. It does not follow links.	https://llmstxt.org
Eval tooling (Ragas, DeepEval, TruLens)	Without it, the choice between all of the above is a guess	https://github.com/confident-ai/deepeval

Smaller members of the "vectorless" family (alphasearch, vectorless-engine, pageindex-go) were only noted, not researched.

5. Recommended build

This section is a recommendation built from the verified facts above. It has not itself been tested.

Step 1: the baseline (an afternoon). Put the whole policy in a cached prompt on a 1M-context model. Tell the model to quote the section and page for every rule and every exception it applies. A 50-page policy costs about \$0.008 per question once cached. Try this first; there is a real chance it is where you stop.

Step 2: only if the documents outgrow the context window or query volume makes cost matter. Build an upgraded RAG:

1. **Split by section, not by size.** Each chunk should be one clause or rule with a stable ID. PageIndex's tree (node_id, title, page range) is one ready-made way to get those section boundaries from a PDF. This use of PageIndex is a suggestion and untested.

2. **Contextual retrieval at ingest.** For a 50-page policy (~25k tokens) this is about \$0.03 at Anthropic's \$1.02 per million tokens.
3. **In the same ingest pass,** have the LLM list the other sections each section refers to, qualifies, or is qualified by ("except as in 7.3", "subject to eligibility"). Store the list as metadata, or write the sections out as an OKF bundle with Markdown links so the result is readable and versioned in git.
4. **Retrieval:** hybrid BM25 + dense search with RRF (Qdrant; free cloud tier or self-hosted), rerank the top 20 to 50 results, keep the top 5 to 8.
5. **Expansion:** add the sections one hop out from the kept results (their listed cross-references), capped at a fixed number.
6. Generate the answer with section and page citations.

Is the hybrid (semantic search to find the entry point, links to pull in context) worth the complexity?

Yes, but only as step 5 above: a one-hop expansion over links you already extracted during ingest. That is about 30 lines of code on top of a good RAG. Do not build an agent that follows links freely: each hop costs a turn, nothing in OKF limits how deep it goes, and Anthropic itself says runtime exploration is slower than precomputed retrieval. Keep the expansion only if the benchmark shows it fixes rule-without-exception answers that reranking alone does not.

A sketch of the expansion over an OKF-style folder (not official code):

```
import posixpath, re, pathlib, yaml

LINK = re.compile(r"\]\\\([^\s#]+\s\.\md)(?:#[^)]*)?\)")

def read_concept(root: pathlib.Path, rel: str):
    _, front, body = (root / rel).read_text(encoding="utf-8").split("---", 2)
    return yaml.safe_load(front), body

def expand(root: pathlib.Path, start: str, hops: int = 1):
    """Start concept plus every concept within `hops` links of it."""
    seen, frontier, out = {start}, [start], []
    for depth in range(hops + 1):
        next_frontier = []
        for rel in frontier:
            meta, body = read_concept(root, rel)
            out.append((depth, rel, meta, body))
            for link in LINK.findall(body):
                target = (link.lstrip("/") if link.startswith("/")
                        else posixpath.normpath(posixpath.join(posixpath.dirname(rel),
link)))
                if target not in seen and (root / target).exists(): # spec tolerates broken
links
                    seen.add(target)
                    next_frontier.append(target)
        frontier = next_frontier
    return out
```

Picks for each part, all verified:

- Framework: LlamaIndex 0.14.25.
 - Embeddings: text-embedding-3-small (\$0.02/1M) or voyage-4-lite (\$0.02/1M, 200M free tokens). To self-host, bge-m3 gives dense and sparse vectors from one model.
 - Vector store: Qdrant.
 - Reranker: Cohere Rerank 4 or Voyage rerank-2.5 (prices unverified).
 - OKF only as the storage format, and only if you want to hand-edit and version the sections.
-

6. Decide it yourself

Candidates. (A) Whole document in a cached prompt. (B) Upgraded RAG, run with and without the one-hop expansion (a test of each layer). (C) PageIndex, self-hosted.

Harness. One Python script that sends every question to each candidate and logs the answer, the retrieved context (or pages cited), the tokens used, the dollar cost and the time taken. Score it with Ragas 0.4.3 (`pip install ragas`) or DeepEval 4.2.6 (`pip install -U deepeval`). Both compute context precision, context recall, faithfulness and answer relevancy. Use a stronger model as the judge than the one that answers, and check 20 of its verdicts by hand.

Question set (50, written by you from your own documents, each with a gold answer and gold page numbers):

- 15 single-rule lookups
- 15 rule plus exception on another page (the case that decides this)
- 10 questions that combine several conditions (eligibility plus exception plus time limit)
- 5 whole-document questions ("list every leave type")
- 5 questions the documents cannot answer (the correct response is to refuse)

Metrics.

- Answer correctness against the gold answer (primary)
- Exception hit rate on the 15 cross-page questions
- Context recall against the gold pages
- Faithfulness
- Correct refusals on the unanswerable questions
- Cost per question
- p50 and p95 latency

Run each candidate 3 times and report the spread between runs.

What settles it. Ship the cheapest candidate that scores within the run-to-run noise of the best on the rule-plus-exception questions and gives no confidently wrong answers on the unanswerable ones. With 15 questions, a gap of 1 question is noise and a gap of 3 or more is real. Keep the one-hop expansion only if B with expansion beats B without it by 3 or more cross-page questions. If A ties everything and your corpus fits, you are done and have built no pipeline.

Public benchmarks for context (the pages were not fetched, so no links): MultiHop-RAG tests evidence spread across documents and is the closest public match to the page-2/page-15 problem. FinanceBench is long filings, and the source of PageIndex's self-reported 98.7%. BEIR compares retrievers on data they were not trained on.

7. Sources

Marks: (vendor) = the vendor reporting on its own product; (third-party) = someone else's measurement; (not independently confirmed) = only seen in search results or blocked.

OKF

- <https://cloud.google.com/blog/products/data-analytics/how-the-open-knowledge-format-can-improve-data-sharing> (the reel's source; v0.1, June 12, 2026)
- <https://cloud.google.com/blog/products/data-analytics/okf-v0-2-adds-trust-signals/> (v0.2, July 24, 2026)
- <https://cloud.google.com/blog/products/data-analytics/scale-okf-bundles-across-an-organization-with-knowledge-catalog> (Knowledge Catalog query path, kcmd push)
- <https://github.com/GoogleCloudPlatform/open-knowledge-format> (licence, stars, README install and enrich commands)
- <https://github.com/GoogleCloudPlatform/open-knowledge-format/blob/main/SPEC.md> (fields, links, conformance)
- <https://github.com/GoogleCloudPlatform/open-knowledge-format/releases> (no releases)
- <https://github.com/GoogleCloudPlatform/open-knowledge-format/commits/main> (last commit Aug 21, 2026)
- https://github.com/GoogleCloudPlatform/open-knowledge-format/tree/main/src/reference_agent
- <https://pypi.org/project/google-okf/> (third-party package)
- <https://pypi.org/project/okflib/> (third-party package)
- <https://en.wikipedia.org/wiki/OKF> (acronym collision)

PageIndex

- <https://github.com/VectifyAI/PageIndex> (README, quickstart, ~\$0.001/page indexing (vendor), 13 s to 4.5 min indexing (vendor))
- <https://api.github.com/repos/VectifyAI/PageIndex> (stars, licence, last push)
- <https://pypi.org/project/pageindex/> (0.2.19)
- <https://pageindex.ai/developer> (Cloud pricing)
- <https://docs.pageindex.ai/getting-started>
- <https://docs.pageindex.ai/tutorials/tree-search/llm> (single-call base algorithm; MCTS note)
- <https://docs.pageindex.ai/endpoints> (citations format)
- <https://docs.pageindex.ai/sdk/documents> (doc_id scope, pagination)
- <https://docs.pageindex.ai/changelog> (Flash, block-level citations)
- <https://pageindex.ai/blog/Mafin2.5> (98.7% FinanceBench, vendor, self-run)

- <https://www.npmjs.com/package/@pageindex/sdk> (fetch blocked; npm version not confirmed)

RAG stack and upgrades

- <https://developers.openai.com/api/docs/pricing> (embedding and cached-input prices)
- <https://developers.openai.com/api/docs/guides/embeddings>
- <https://docs.voyageai.com/docs/pricing>
- <https://docs.voyageai.com/docs/reranker> (models; price not shown)
- <https://docs.cohere.com/docs/cohere-embed>
- <https://cohere.com/pricing> (dedicated-instance prices only; per-search price not confirmed)
- <https://huggingface.co/BAAI/bge-m3>
- <https://huggingface.co/Qwen/Qwen3-Embedding-8B> (MTEB rank, vendor)
- <https://huggingface.co/spaces/mteb/leaderboard> (table not loadable; rankings not confirmed)
- <https://github.com/pgvector/pgvector>
- <https://github.com/qdrant/qdrant>
- <https://qdrant.tech/pricing/>
- <https://qdrant.tech/documentation/concepts/hybrid-queries/>
- <https://weaviate.io/pricing>
- <https://www.pinecone.io/pricing/>
- <https://www.trychroma.com/pricing>
- <https://github.com/facebookresearch/faiss>
- <https://docs.pinecone.io/guides/data/filter-with-metadata>
- https://reference.langchain.com/python/langchain-classic/retrievers/multi_query/MultiQueryRetriever
- <https://pypi.org/project/langchain/>
- <https://pypi.org/project/llama-index/>
- <https://pypi.org/project/haystack-ai/>
- https://developers.llamaindex.ai/python/framework/getting_started/starter_example/
- <https://www.anthropic.com/news/contextual-retrieval> (failure-rate reductions and \$1.02/1M, vendor)
- <https://github.com/stanford-futuredata/ColBERT>
- <https://github.com/illuin-tech/colpali> (deprecation notice; ViDoRe scores, from the model authors)
- <https://www.microsoft.com/en-us/research/blog/graphrag-new-tool-for-complex-data-discovery-now-on-github/> (naive RAG fails on whole-corpus questions)

Graph and hierarchy

- <https://github.com/microsoft/graphrag> (maintenance mode, cost warning)
- <https://pypi.org/project/graphrag/> (3.2.0)
- <https://microsoft.github.io/graphrag/>
- <https://microsoft.github.io/graphrag/query/overview/>
- <https://microsoft.github.io/graphrag/cli/>
- <https://theneuralbase.com/graphrag/learn/beginner/cost-comparison-graphrag-is-expensive/> (third-party)

- <https://www.falkordb.com/blog/reduce-graphrag-indexing-costs/> (third-party)
- <https://aws.amazon.com/blogs/opensource/unified-knowledge-graph-rag-on-aws-graphrag-and-lightrag-on-one-stack/> (third-party cost, latency and F1)
- <https://medium.com/graph-praxis/the-graphrag-cost-cliff-how-33-000-became-33-in-eighteen-months-be1b0fbe37e4> (not independently confirmed; fetch returned 403)
- <https://github.com/HKUDS/LightRAG>
- https://github.com/HKUDS/LightRAG/blob/main/examples/lightrag_openai_demo.py
- <https://pypi.org/project/lightrag-hku/> (1.5.7)
- <https://arxiv.org/abs/2410.05779>
- <https://github.com/parthsarathi03/raptor>
- <https://github.com/parthsarathi03/raptor/commits/master> (last commit Sep 3, 2024)
- RAPTOR paper, arXiv:2401.18059 (the PDF did not render when fetched; paper claims here come from the repo and summaries)
- <https://github.com/getzep/graphiti>
- <https://github.com/getzep/graphiti/tree/main/examples/quickstart>
- <https://pypi.org/project/graphiti-core/>
- <https://www.getzep.com/pricing>
- <https://github.com/topoteretes/cognee>
- <https://pypi.org/project/cognee/>
- <https://www.cognee.ai/pricing>
- <https://docs.cognee.ai/quickstart>

Agent-native, long context, evaluation

- <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- <https://www.llamaindex.ai/blog/files-are-all-you-need> (vendor blog)
- <https://www.llamaindex.ai/blog/is-grep-all-you-need-lexical-vs-semantic-search-for-agents> (vendor blog)
- <https://llmstxt.org>
- <https://modelcontextprotocol.io>
- <https://modelcontextprotocol.io/specification/latest> (2026-07-28)
- <https://modelcontextprotocol.io/docs/sdk>
- <https://platform.claude.com/docs/en/models/overview> (context windows)
- <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
- <https://claude.com/pricing>
- <https://developers.openai.com/api/docs/models>
- <https://ai.google.dev/gemini-api/docs/gemini-3>
- <https://ai.google.dev/gemini-api/docs/caching>
- <https://ai.google.dev/gemini-api/docs/pricing>
- <https://pypi.org/project/ragas/> (0.4.3)
- <https://pypi.org/project/deepeval/> (4.2.6)

- <https://github.com/confident-ai/deepeval>
- <https://pypi.org/project/trulens/> (2.14.0)
- Cache-augmented generation paper, arXiv:2412.15605; MultiHop-RAG; FinanceBench; BEIR (found through search only; not fetched, so not linked)
- Claude Code dropping RAG for agentic search: only in secondary write-ups and public statements, not confirmed from a primary source